

Qiyas Data Science / AI Hackathon — Instructions

Ethiopian Smallholder Crop-Yield Challenge · Full-day hackathon · Qiyas / IADE Training Program, Addis Ababa University

Oct 5, 2026 · Qiyas Team

1. The Challenge

You have survey data from smallholder farm plots across five Ethiopian regions and five crops. Your job: predict `yield_tons_per_ha` for each plot in the test set, using only information known at or before planting time. You'll be scored on how close your predictions are to the true yields, which only the organizers have.

The plot data alone isn't the whole picture. You're also given a regional weather table and a market price table. Part of the challenge is figuring out how to combine all three correctly — the weather table, in particular, contains real signal that a plot-level model alone can't see.

All three datasets are raw exports — real-world messy, not cleaned for you. Missing values, inconsistent spelling, impossible values, duplicate rows, a unit mix-up. Cleaning and joining them correctly is part of the task, and it's graded.

This is a full data-science workflow, not just a modeling contest. You will integrate the data, analyze it, visualize it, build and rigorously evaluate a model, and ship it as a working demo. Seven deliverables (A–G, Section 4) plus your prediction file make up your grade.

Key idea: this is a regression task, not classification: you're predicting a number (tons/ha), not a category. Different loss (MSE/MAE, not cross-entropy), different output layer (linear, not softmax), different metrics (RMSE/R², not accuracy).

2. The Data (three tables)

2.1 Plot-level survey data (the core table)

File	Rows	Contents
crop_yield_train.csv	15,090	Full feature set + true yield_tons_per_ha. Explore, clean, and train on this.
crop_yield_leaderboard_test.csv	3,750	Same features, no yield column. These are the plots you predict.
submission_template.csv	3,750	plot_id + predicted_yield_tons_per_ha, plot_ids pre-filled in order. Fill in and submit this.

Data dictionary — plot-level

Column	Type	Description
plot_id	string	Unique plot ID. Join key for scoring.
region	categorical	Oromia / Amhara / SNNPR / Tigray / Somali.
crop_type	categorical	teff / wheat / maize / sorghum / barley.
survey_year	integer	2021-2024.
planting_month	categorical	Month the crop was planted.
altitude_m	numeric	Elevation, meters above sea level.
rainfall_mm_season	numeric	Plot-level self-reported growing-season rainfall, mm.
farm_size_ha	numeric	Farm size, hectares.
fertilizer_kg_per_ha	numeric	Fertilizer applied, kg/ha.
improved_seed_used	binary (0/1)	Certified improved seed used?
pest_disease_flag	binary (0/1)	Pest/disease pressure observed?
soil_quality_index	numeric (0-1)	Composite soil quality score.
labor_days_per_ha	numeric	Labor-days invested per hectare.

Column	Type	Description
distance_to_market_km	numeric	Distance to nearest market.
yield_tons_per_ha	numeric (target)	TRAIN ONLY. What you're predicting.

2.2 Regional weather data — regional_weather.csv (join this in)

A separate table of monthly weather readings by region and year — not merged in for you. It's one row per region + year + month (about 230 rows after the usual real-world gaps).

Column	Type	Description
region	categorical	Join key. Formatted inconsistently — see below.
year	integer	Join key, matches survey_year.
month	categorical	Jan-Dec.
avg_temp_c	numeric	Average temperature that month, °C.
monthly_rainfall_mm	numeric	Total rainfall that month, mm.
extreme_heat_days	integer	Count of unusually hot days that month.

This is a join, not a lookup. This table is monthly; the plot data is seasonal. You'll need to decide which months count as "the growing season" for a given plot and aggregate yourself (a common starting point: the planting month plus the following 3 months). Region names here are also formatted inconsistently — and not in the same way as the plot table's region column, so fixing one doesn't automatically fix the other. Expect some missing region/year/months and a few duplicate readings too.

2.3 Market price data — market_prices.csv (input to your analysis and demo, NOT a model feature)

Crop prices by region and year. About 100 rows (5 crops × 5 regions × 4 years).

Column	Type	Description
crop_type	categorical	Join key.

Column	Type	Description
region	categorical	Join key.
year	integer	Join key.
price_birr_per_quintal	numeric	Price in Ethiopian birr per quintal (1 quintal = 100 kg; 1 ton = 10 quintals).

Price is for analysis and the demo, not the model. Use this table to turn yield into revenue (price × yield) in your analysis and in your deployed demo — it does NOT belong as an input feature to your yield model. Price doesn't cause yield; feeding it in is a logical error (and a form of leakage), not a free accuracy boost. A few rows in this table also look like they're recorded in the wrong unit — worth a sanity check before you trust it.

3. Rules

1. Teams of 3–4, assigned at the start of the day.
2. Any library, any model type — linear, trees/forests, neural nets, ensembles.
3. AI coding assistants are allowed and encouraged. This hackathon tests judgment and iteration, not typing speed.
4. Don't reverse-engineer true yields from the test file, and don't share code or predictions across teams.
5. Your final model must use at least one feature derived from the weather table (Section 2.2) — a plain re-use of rainfall_mm_season alone doesn't count.
6. Learn everything from the train file only: medians for imputing, outlier caps, category lists, scalers and encoders are fitted on train and then applied to test. Never fit anything on the test file.
7. Every number, table and figure you report must be produced by code in your project folder — no hand-edited CSVs, no screenshots of spreadsheets.
8. You submit once, at the end of the day: one project folder in the layout shown in Section 6, containing your submission file and all seven deliverables.
9. All seven deliverables (Section 4) are required. Skipped items score zero, regardless of your prediction score.

Suggested team roles. With 3–4 people and one day, split the work in parallel: a data lead (Deliverable A), an analysis and visualization lead (B and C), a modeling lead (D), and a deployment lead (E, plus the folder and README in G). Everyone reviews the slides (F). The pipeline from A feeds everything else, so agree early on the exact column names of your master tables.

3.1 Recommended language & libraries

Use **Python 3.10 or newer**. Everything in this assignment — cleaning, joins, plots, models, the demo — has a mature Python libraries. Work in Jupyter notebooks (JupyterLab, VS Code, or Google Colab). You may use other libraries, but anything outside this list is at your own risk, and you must list every package in your requirements.txt.

Job	Recommended	Also fine / notes
Data wrangling & joins (A, B)	pandas, numpy	polars if your team already knows it. Use pandas string methods (str.strip, str.lower, replace) for label cleaning.
Integrity checks (A7)	plain assert statements	pandera for schema checks (optional).
Static figures (C)	matplotlib + seaborn	Save PNGs with fig.savefig(..., dpi=150, bbox_inches="tight"). missingno is handy for fig01.
Interactive charts	plotly	Static export needs the kaleido package. Needed for the Interactive explorer stretch goal.
Join diagram (A3)	graphviz, draw.io, or matplotlib	Any tool is fine as long as the diagram appears in your report.

Job	Recommended	Also fine / notes
Preprocessing & pipelines (A, D)	scikit-learn: Pipeline, ColumnTransformer, SimpleImputer, OneHotEncoder	A Pipeline keeps train and test steps identical and makes Rule 6 almost automatic.
Baselines & linear models (D1)	scikit-learn: DummyRegressor, LinearRegression, Ridge	
Tree & boosting models (D2)	scikit-learn: RandomForestRegressor, HistGradientBoostingRegressor; also XGBoost, LightGBM	On tabular data like this, boosted trees usually beat neural networks.
Neural network (optional model family)	scikit-learn MLPRegressor, or PyTorch / Keras	Only if you have time; expect more tuning effort for similar or worse accuracy.
Evaluation (D1, D3, D4, D5)	scikit-learn: train_test_split, KFold / cross_val_score, mean_squared_error, mean_absolute_error, r2_score	Always set random_state.
Tuning (D6)	GridSearchCV or RandomizedSearchCV	Optuna for larger searches.
Feature importance (fig12)	sklearn.inspection.permutation_importance	SHAP is optional and slower.
Saving the model (E)	joblib	Save the whole Pipeline (preprocessing + model) so the demo applies the same steps.
Demo app (E)	Streamlit or Gradio	Flask / FastAPI if you want a custom HTML form.
Hosting (E)	Hugging Face Spaces or Streamlit Community Cloud	Local demo is accepted (see Deliverable E).
Reports (A, B, D)	Jupyter Notebook → HTML/PDF via nbconvert	Markdown or Word also accepted.

Job	Recommended	Also fine / notes
Slides (F)	PowerPoint or Google Slides	Export to PDF.
Team collaboration	Git + GitHub, or a shared Google Drive / Colab folder	Agree who owns which notebook so you don't overwrite each other.

Practical tips

- Put cleaning and modeling in a single scikit-learn Pipeline, so the exact steps fitted on train run unchanged on test.
- Pin versions in requirements.txt. The demo must run on a different machine (or on Hugging Face / Streamlit Cloud) than the one you trained on.
- Don't use Excel or Google Sheets to clean or edit data. Rule 7 requires everything to come from code.
- Install only what you need. Every extra package is one more thing that can fail on the day.
- Prefer one library per job: pick one of matplotlib/seaborn or plotly as your main plotting tool so the figure pack looks consistent.

4. Deliverables (all seven required)

Here is the full list with the weight of each part of your grade. Full scoring criteria are in the separate Judging Rubric.

Part	What you produce	Points
Prediction score	Your completed submission file, scored against the hidden answer key.	20
A — Data Cleaning & Integration Pipeline	Cleaned, joined, feature-engineered master tables; cleaning log; join map and audit; integrity checks.	14
B — Data Analysis Report	14 numbered analysis tasks across prices, categories, agronomic drivers, and time/weather.	14
C — Visualization Pack	12 required figures, each with a one-line takeaway.	14

Part	What you produce	Points
D — Modeling & Evaluation	Model comparison, cross-validation, out-of-time check, weather ablation, tuning, error analysis.	14
E — Deployed Prediction Demo	A working app that looks up weather and price for you and returns yield and revenue.	8
F — 5-Slide Presentation	5 minutes + 2 minutes Q&A, with a live demo.	6
G — Project Structure & Reproducibility	Folder layout, README, requirements file, valid submission.	5
Stretch goal (optional)	Pick one of four.	5
TOTAL		100

Grading works the same way for every item: a specific number, table or chart plus a one- or two-sentence interpretation earns full credit. A result with no interpretation earns half. A correctly-reasoned "we found nothing here, and here's how we know" earns full credit — you don't have to find something dramatic to be right. A skipped item earns zero.

A — Data Cleaning & Integration Pipeline (14 pts)

Turn three raw exports into one trustworthy modeling table — and prove it's trustworthy. Everything here must be done in code, so the exact same pipeline can be run on the test file.

- **A1 Cleaning log:** one table, one row per issue — file, column(s), issue type, how many rows affected (count and %), the fix you applied, and why that fix. Cover all three files: at least 4 distinct issues in the plot file, 2 in the weather file, and 2 in the price file.
- **A2 Key standardization proof:** for region (in all three tables) and crop_type (plot and price tables), show the unique values before and after cleaning. After cleaning, every table must use the identical set of region labels, and the plot and price tables the identical set of crop labels.
- **A3 Join map & diagram:** a diagram of how the three tables connect (draw it in any tool) showing the join keys, join type, expected cardinality (one-to-one or many-to-one), and your growing-season window rule — plus a short sentence on why you chose which table as the "left" table.

- **A4 Join audit:** for each join (plot → weather, plot → price) report the match rate, the row count before and after the join (it must not change for a many-to-one join — if it does, find and fix the duplicate keys), how many plots went unmatched and why, and what you did about them. For the weather join also report how many plots had fewer than a full set of months in their growing-season window and how you handled that.
- **A5 Join proof:** pick 3 specific plots and show your work — which weather rows did you pull for each one's growing-season window, and what season-level number did they produce? (This proves the join logic is right, not just that the code ran.)
- **A6 Feature engineering table:** at least 6 engineered features — at least 3 derived from weather (e.g. growing-season mean temperature, season rainfall total, count of extreme-heat days, deviation from the region's typical temperature), at least 1 ratio or interaction (e.g. fertilizer × improved seed), and at least 1 from planting_month. For each: name, formula, source columns, and why you expect it to help.
- **A7 Integrity checks:** at least 5 automated checks written as code (assert statements or a validate() function) that run on your master tables and print PASS/FAIL — for example: plot_id is unique, row count unchanged by the joins, no missing values in model features, region and crop labels all in the allowed set, values in plausible ranges (yield ≥ 0, price > 0), and train and test have the same feature columns. Include the printed output in your report.
- **A8 Master tables & data dictionary:** export master_train.csv and master_test.csv (cleaned + joined + engineered, same feature columns, test without the target), plus data_dictionary_master.csv listing every column with its type, source file, description, and how it was derived. The test file must go through the same pipeline as train, using statistics learned from train only (Rule 6).

B — Data Analysis Report (14 pts)

One document (a notebook exported to HTML/PDF, or Markdown/Word/PDF) with each task under its own heading showing its ID. Answer every task with an actual number, table, or chart, plus one or two sentences of interpretation. Use your cleaned, joined data from Deliverable A. 1 point each.

B1 — Value & prices

- **B1.1 Revenue per hectare by crop:** compute estimated revenue per hectare (yield_tons_per_ha × 10 quintals per ton × price_birr_per_quintal) for each crop. Which crop earns the most per hectare? Is it the same crop that has the highest yield?

- **B1.2 Price trend:** how did each crop's price change from 2021 to 2024 (absolute and %)? Which grew fastest, and is the gap between the fastest and slowest large enough to matter?

B2 — Category breakdowns

- **B2.1 Yield by region:** mean, median, and standard deviation of yield_tons_per_ha per region. Which region has the highest average? The most variable?
- **B2.2 Yield by crop type:** same breakdown by crop. Does the ranking match what you'd expect from each crop's inherent yield potential, or is something surprising?
- **B2.3 Region × crop pivot table:** mean yield for every region/crop combination. Which single combination performs best? Which worst? Give one plausible agronomic reason for the pattern.
- **B2.4 Improved seed, by crop:** the yield gap with vs. without improved seed, split by crop — in tons/ha and as a % lift. Which crop benefits most, and is the % lift consistent across crops?
- **B2.5 Pest/disease, by crop:** the yield gap with vs. without the pest/disease flag, split by crop — in tons/ha and as a %. Which crop is hit hardest in absolute and in relative terms?

B3 — Agronomic drivers

- **B3.1 Fertilizer response:** bin fertilizer_kg_per_ha into at least 4 bands and show mean yield per band, overall and per crop. Is the response linear, flattening (diminishing returns), or flat?
- **B3.2 Altitude by crop:** bin altitude_m into at least 4 bands. For each crop, which band gives the highest mean yield? Flag any band/crop cell with too few plots (under 30) to trust.
- **B3.3 Planting month:** mean yield by planting month, overall and per crop. Does timing matter — and how big is the effect in tons/ha compared with the region and crop effects you found above?
- **B3.4 Distance to market:** is distance_to_market_km related to yield? Report a correlation and a binned comparison, then say whether you'd keep it as a model feature and why.

B4 — Time & weather

- **B4.1 Yield trend:** mean yield by year 2021–2024, overall and by region. Is there a real trend or does it just bounce around? State which, and how you decided (e.g. compare the size of year-to-year changes with the spread of yields).

- **B4.2 Weather anomalies:** for each region-year, compute the growing-season mean temperature from your join and its deviation from that region's own 4-year average. Which region-year is most unusual? Compare yield in that region-year with the same region's other years — and check whether the effect is the same for every crop.
- **B4.3 Plot-reported vs. station rainfall:** compare each plot's own rainfall_mm_season with the rainfall you aggregate from regional_weather.csv for the same region and season. Do they agree? Quantify it (correlation, mean absolute difference, and the typical level of each) and suggest why they might differ.

C — Visualization Pack (14 pts)

Produce the 12 figures below, one file per figure, saved as PNG (at least 1200 pixels wide, or 150 dpi) in your figures/ folder under the exact file names shown. Use any plotting library (matplotlib, seaborn, plotly with static export, etc.), but every figure must be generated by code in your project. 1 point per figure, plus 2 points for overall figure quality.

File name	What it must show
fig01_missingness.png	Share of missing or invalid values (count -999 sentinels) in every column, for all three raw tables.
fig02_before_after_cleaning.png	Before-and-after distributions (histogram or box plot) for at least 2 variables where cleaning visibly changed the picture, e.g. farm size, fertilizer, price per quintal.
fig03_yield_distribution.png	Distribution of yield_tons_per_ha overall and by crop type (histogram, violin, or box).
fig04_region_crop_heatmap.png	Annotated heatmap of mean yield by region × crop (your B2.3 pivot, visualized), with plot counts shown or noted.
fig05_correlation_heatmap.png	Correlation heatmap of yield, all numeric plot features, and your weather-derived features.
fig06_climate_by_region.png	Monthly average temperature (and/or rainfall) across the year for each region, with the growing-season window for at least one planting month shaded.

File name	What it must show
fig07_yield_vs_season_temp.png	Yield against growing-season mean temperature, one panel per crop, with a trend line or binned means — does each crop have a sweet spot?
fig08_price_trends.png	Price per quintal by year for each crop (unit-corrected), by region or averaged — your B1.2 story in one picture.
fig09_revenue_by_crop_region.png	Revenue per hectare by crop and region (grouped bars or heatmap) — your B1.1 story with the region dimension added.
fig10_model_comparison.png	Validation RMSE of every model you tried (bars), with the mean-predictor baseline marked and cross-validation error bars on at least your final model.
fig11_predicted_vs_actual_residuals.png	Two panels: predicted vs. actual yield with a $y = x$ reference line, and residuals vs. predicted.
fig12_feature_importance.png	Importance of your top 10+ features (permutation importance recommended), with weather-derived features visually highlighted.

Figure quality (applies to every figure)

- Every figure has a title, labeled axes with units, a legend whenever there is more than one series, and fonts readable at slide size.
- One consistent visual style across the pack, and a colorblind-safe palette.
- Honest axes: bar charts start at zero unless clearly noted, and no chart is cropped or scaled to exaggerate a difference.
- A figures/figure_captions.md file with a one- or two-sentence caption per figure: what it shows, and the takeaway ("so what").

D — Modeling & Evaluation (14 pts)

Report results in one document (notebook plus a short written summary is fine). Every score is computed on held-out data from the train file — never the test file. State your split (size and random seed).

- **D1 Baselines:** a mean-predictor baseline and a simple linear model, each with RMSE, MAE, and R^2 on a held-out validation split.
- **D2 Model comparison:** at least 3 model families beyond the baselines (e.g. random forest, gradient boosting such as XGBoost/LightGBM, a neural network), compared in one table on the same split and features, with RMSE, MAE, R^2 , and training time. Name your winner and explain why you chose it.
- **D3 Cross-validation:** at least 5-fold cross-validation for your final model and one runner-up, reported as mean $\pm$ standard deviation of RMSE. Does it agree with your single-split score? What does the spread tell you about reliability?
- **D4 Out-of-time check:** train on 2021–2023 and validate on 2024 (or leave-one-year-out). Compare with your random-split score and explain any gap, or the absence of one.
- **D5 Weather ablation:** train your final model type twice on identical splits — once without any weather-derived features, once with them — and report RMSE, MAE, and R^2 for both and the difference. Was the weather join worth the effort, and by how much?
- **D6 Tuning:** a documented hyperparameter search (grid, random, or Optuna) for your final model — search space, number of trials, best parameters, and score before vs. after. Tune on train/validation or cross-validation only.
- **D7 Error analysis:** (a) error by crop type and by region (table or chart); (b) error against at least one continuous feature — is it worse at extremes such as high altitude or unusual temperature?; (c) the 10 largest-error validation plots listed, with a hypothesis for why each group is hard.
- **D8 Response to findings:** what you changed because of D7, and whether it helped. An honest "it didn't help" earns full credit.
- **D9 Plain-language metric:** translate your final RMSE and MAE into (i) tons/ha and as a % of mean yield, and (ii) approximate birr per hectare of error (use the average price) — one short paragraph a farm cooperative manager could follow.

E — Deployed Prediction Demo (8 pts)

A small, working interface for your final model — not just a notebook cell. Someone who has never seen your code should be able to enter a plot's details and get back a predicted yield and an estimated revenue.

- **Inputs:** the user chooses region, crop type, survey year (2021–2024) and planting month, and enters the plot details (altitude, farm size, fertilizer, seed, pest flag, soil quality, labor days, distance to market).
- **Integration:** the user must NOT type in any weather numbers or prices. The app looks up the growing-season weather features and the price for the chosen region/crop/year automatically, from your cleaned tables bundled with the app (e.g. app/assets/).
- **Outputs:** predicted yield (tons/ha) and estimated revenue (predicted yield × farm size × 10 × price, in birr), plus a line showing which weather and price values were looked up (e.g. "season average 18.2 °C, price 4,950 birr/quintal").
- At least one visual in the app — for example your prediction next to the average yield for that region and crop, or a what-if chart of predicted yield across fertilizer amounts.
- Build it with whatever is fastest for you: Gradio, Streamlit, or a small Flask/FastAPI app with a basic HTML form — AI coding assistants are very good at scaffolding any of these quickly.
- Host it somewhere reachable by a URL (e.g. Hugging Face Spaces or Streamlit Community Cloud — both have free tiers). No internet or hosting not possible on the day? A local demo is fine — one command to run it (e.g. streamlit run app/app.py), demoed live from your laptop.
- Handle normal inputs without crashing, and give a friendly message for empty or nonsensical ones. It does not need to be right on bizarre inputs, just not broken.

F — 5-Slide Presentation (6 pts) — 5 min + 2 min Q&A

- Slide 1 — the problem and the three datasets, in your own words. Slide 2 — cleaning & integration: your join map and the key numbers from A4. Slide 3 — what the data says: your two most interesting findings from B/C, with figures. Slide 4 — modeling & evaluation: model comparison, weather ablation, cross-validation. Slide 5 — error analysis, your final score, demo link, and what you'd try next.
- Use at least 2 figures from your Visualization Pack on the slides.
- Be ready to open your deployed demo live and run a prediction (with revenue estimate) during Q&A.

G — Project Structure & Reproducibility (5 pts)

- **G1 Folder structure:** your project follows the layout in Section 6, with deliverables in the expected places and files named as shown.

- **G2 README.md:** team name and members, a one-paragraph summary, exact setup and run commands, the order to run the notebooks, where each deliverable lives, your final validation score, and your demo link (or a note that it runs locally).
- **G3 Reproducibility:** a requirements.txt (or environment.yml) with versions, relative file paths only, fixed random seeds, and notebooks that run top to bottom in a fresh environment.
- **G4 Valid submission file:** correct name, exactly the columns plot_id and predicted_yield_tons_per_ha, all 3,750 plot_ids once each, in the original order, no blank or non-numeric predictions.
- **G5 Method hygiene:** the final model uses at least one weather-derived feature (Rule 5), nothing was fitted on the test file (Rule 6), and price is not used as a model feature.

5. Stretch Goal (optional, pick one — 5 pts)

- **Interactive explorer:** a dashboard (Plotly Dash, Streamlit, or interactive Plotly HTML) with filters for region, crop, and year that updates at least 3 charts.
- **Regional equity:** does your model perform worse in any region? Would that under-serve it if used for real fertilizer allocation? Propose one mitigation.
- **Uncertainty estimation:** produce a prediction interval per plot (bootstrapping or quantile regression) and explain what it means for a decision-maker.
- **Monitoring & drift memo:** one page — how would you know if this model's predictions degrade over time (new seasons, shifting rainfall/temperature patterns)? What would you monitor, and when would you retrain?

6. Submission & Folder Structure

You submit once, at the end of the day, a public Git repository link. Use exactly this layout so judges can find everything easier.

6.1 Folder layout

```
team_<NAME>/
+-- README.md          setup, run order, summary, demo
link    (G2)
+-- requirements.txt    pinned package versions
(G3)
|
```

```

+-- submission/
|   +-- team_<NAME>_submission.csv           the file that gets scored
(G4)
|
+-- data/
|   +-- raw/                                the original 5 CSVs, never edited
|   +-- processed/
|       +-- master_train.csv
(A8)
|       +-- master_test.csv
(A8)
|       +-- data_dictionary_master.csv
(A8)
|
+-- notebooks/                               (or one notebook with clear A-D
headings)
|   +-- 01_cleaning_and_integration.ipynb
(A)
|   +-- 02_analysis_report.ipynb
(B)
|   +-- 03_visualizations.ipynb
(C)
|   +-- 04_modeling_and_evaluation.ipynb
(D)
|
+-- src/                                     optional: reusable pipeline code
|   +-- cleaning.py   features.py   train.py   predict.py
|
+-- models/
|   +-- final_model.joblib                 the trained model the demo loads
|
+-- figures/
|   +-- fig01_missingness.png ... fig12_feature_importance.png
(C)
|   +-- figure_captions.md
(C)
|
+-- reports/
|   +-- A_cleaning_and_integration.(pdf|md|html)   log, join map, audit,
proof   (A)
|   +-- B_analysis_report.(pdf|md|html)
(B)
|   +-- D_model_evaluation.(pdf|md|html)

```

```

(D)
|
+-- app/
(E)
|   +-- app.py           Streamlit / Gradio / Flask entry
point
|   +-- requirements.txt
|   +-- assets/         cleaned weather + price tables,
model
|
+-- presentation/
    +-- team_<NAME>_slides.(pdf|pptx)
(F)

```

6.2 Where each deliverable goes

Deliverable	What judges will look for	Location
Prediction score	The scored file	submission/team_<NAME>_submission.csv
A — Pipeline	Code, written log/map/audit/proof/feature table/check output, and the three exported data files	notebooks/01_..., reports/A_..., data/processed/
B — Analysis	All 14 tasks, each under its ID heading	reports/B_analysis_report.*, notebooks/02_...
C — Visualizations	12 PNGs with the exact names, plus captions	figures/, notebooks/03_...
D — Modeling	D1-D9 write-up, code that reproduces it, the saved final model	reports/D_..., notebooks/04_..., models/
E — Demo	App code, bundled lookup tables, and the demo URL	app/, URL in README.md
F — Slides	Your 5 slides	presentation/
G — Structure	README, requirements, folder layout	project root

6.3 Naming and hygiene rules

- Lowercase file and folder names with underscores — no spaces, no special characters. Number your notebooks (01_, 02_, ...) in run order.
- Relative paths only: code must never contain a path like C:\Users... or /content/drive/... Anything it reads should live inside the project folder.
- Never edit files in data/raw/. All cleaning happens in code that writes to data/processed/.
- Reports can be PDF, HTML, Markdown, or Word. A notebook exported to HTML counts, as long as each task or figure sits under its ID heading (A1, B2.3, D5, ...).
- One notebook instead of four is fine if its sections are clearly headed with the deliverable IDs — but the folders, figures, README, and submission file stay as shown.
- Leave out virtual environments (venv/, .venv/), **pycache/**, .ipynb_checkpoints/, and any file over 100 MB.

6.4 Pre-submission checklist

- ☐ Submission CSV opens, has 3,750 rows, no blank predictions, no duplicate plot_ids.
- ☐ All 12 figures exist under the exact names, and figure_captions.md has 12 entries.
- ☐ master_train.csv, master_test.csv and data_dictionary_master.csv exist in data/processed/.
- ☐ Notebooks run top to bottom from a fresh environment with the commands in your README.
- ☐ Your demo URL works in a private/incognito window — or the one-line local run command is in the README.
- ☐ Slides are in presentation/ and you have rehearsed the live demo.

7. FAQ

Can we drop a column we think isn't useful? Yes — that's a legitimate, gradeable decision, as long as you investigated it first (and say so in your report).

A row looks impossible — can we drop or cap it? Yes. State what you did and why in your cleaning log (A1). Reasonable judgment calls aren't penalized. Remember that the test file has similar problems and you still have to predict every plot in it, so think about how your pipeline handles bad values rather than just deleting rows.

How exactly should we define "growing season" for the weather join? We won't hand you an exact answer — but a reasonable starting point is the planting month plus the following three months. Document whatever window you choose and why; you won't be marked down for a sensible choice that differs from another team's.

Do we really need all 12 figures and all 14 analysis tasks if time is short? Yes — every item is graded and a skipped item scores zero. That's why the work is built to be split across a team. Quick, correct and clearly interpreted beats elaborate but unfinished.

Can we use one notebook instead of four? Yes, as long as each deliverable's section is headed with its ID and the folder layout in Section 6 is otherwise followed.

We can't get public hosting working — what now? Run it locally and demo it live from your own machine during your presentation. Say so in your README and on your final slide — you won't be penalized for a hosting issue outside your control, only for not having a working demo at all.

How exactly is our prediction score calculated? It's based on the RMSE of your submission against the hidden answer key, scaled between a "predict the average" floor and a strong-reference ceiling. See the Judging Rubric for the formula.

Good Luck!